

Better Lookups for `map`, `unordered_map`, and `flat_map`

Document #: P3091R4
Date: 2025-06-20 10:35 EEST
Project: Programming Language C++
Audience: LEWG
Reply-to: Pablo Halpern
<phalpern@halpernwrightsoftware.com>

Contents

- 1 Abstract
- 2 Change Log
- 3 Motivation
- 4 Proposed Feature
 - 4.1 Overview
 - 4.2 Not Proposed: Enhancements to `optional::value_or`
- 5 Before and After Comparisons
- 6 Alternatives Considered
 - 6.1 Other Names for `get`
 - 6.2 Build Get-Value-or-Return-Alternative Functionality Directly into `map`
 - 6.3 Free Functions Instead of Members
- 7 Implementation Experience
- 8 Wording
 - 8.1 Feature-Test Macros
 - 8.2 `std::map` Changes
 - 8.3 `std::unordered_map` Changes
 - 8.4 `std::flat_map` Changes
- 9 Acknowledgments
- 10 References

§ 1 Abstract

The most convenient way to look up an element of a map-like container is to use the index operator, i.e., `theMap[key]`. This operator cannot be used, however, when 1) the container is `const`, 2) the

mapped type is not default constructible, 3) the default-constructed value is inappropriate for the context, or 4) the container should not be modified. These limitations often force the user to resort to the `find` member function, which returns an iterator that points to a pair and typically leads to more complex code having at least one `if` statement and/or duplicate lookup operations. Taking inspiration from other languages, especially Python, this paper proposes a `get` member function that returns `optional<T&>` and leverages the observers and monadic operations of `optional` to simplify lookups for `map`, `unordered_map`, and `flat_map`. Note that this proposal is built on `optional<T&>`, which is described in [P2988R12] and is expected to be voted into the C++ Working Draft in Sofia. It's usefulness is also enhanced by the `maybe` facilities described in [P1255].

§ 2 Change Log

R4 (2025-06-20 in Sofia)

- Rebased to [N5008]
- Added `constexpr`.
- Added `flat_map` support.
- Fixed typo in `feature-test` macro.
- Recorded the vote in LEWG to keep the name `.get` and not change it to `.lookup`.

R3 (2024-10-15 pre-Wroclaw mailing)

- Removed wording that duplicates parts of [P2988R7].
- Removed enhancements to `optional<T>::value_or` that will be handled in a separate paper, [P1255], by Steve Downey.

R2 (2024-05-23 pre-St. Louis mailing)

- LEWG Reached consensus on some aspects of [P2988R4] - specifically that `value_or` should return an rvalue for both `optional<T>` and `optional<T&>`. This change is reflected in the wording for this paper. Rather than a separate `optional<T>::or_construct` member, the functionality of `or_construct` is folded into an enhanced `value_or` both `optional<T>` and `optional<T&>`.

R1 (post 2024-02-27 LEWGI teleconference)

- As a result of discussions in LEWGI on 27 Feb 2024, the return value of `get` was changed from `mapped_type` to `optional<mapped_type&>`, and the functionality of `get_ref` and `get_as` were delegated to `optional`. Note that `optional<T&>` is not valid in C++23 but is proposed in [P2988R4] for C++26.
- Added a proposed `or_construct` member to `optional`
- Proposed an enhancement of `optional<T&>::value_or` over that in [P2988R4]
- Added `feature-test` macros

R0 (2024-02-15 pre-Tokyo mailing)

- Initial version

§ 3 Motivation

Just about every modern computer language has, as part of the language or its standard library, one or more associative containers that uniquely map a key to a value, variously called `map`, `hash_map`, `dictionary`, or something similar. In C++26, we have `std::map`, `std::unordered_map`, and `std::flat_map`. The easy-to-write and easy-to-read expression for getting a value for an associated key is simply `theMap[key]`; in other words, a mapped value is retrieved (and often set) using the index operator, which returns a reference to the value associated with the key. Unfortunately, the index operator in the C++ associative containers has a number of shortcomings compared to many other languages.

- It works only for non-`const` containers.
- It works only for default-constructible mapped types.
- The default-constructed object (when available) is not always the correct identity for a given use case.
- It modifies the container when the key is absent.

Consider a `std::unordered_map` named `theMap` that maps an integer key to a floating-point value, modeling a sparse array of `double`. If we want to find the largest of the `double` values mapped to the integer keys in the range 1 to 100, we might be tempted to write the following loop:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
    largest = std::max(largest, theMap[i]);
```

If `theMap` is `const`, the loop will not compile. If any of the keys in the range `[1, 100]` are absent from the map, then `theMap[i]` will return a default-constructed `double` having value 0.0, which might or might not be desirable, depending on whether we want to treat missing values as truly having value 0.0 or to ignore them (or, equivalently, to treat them as having value $-\infty$). Finally if, before executing this loop, `theMap` contains only, say, five entries, then at the end of the loop, it will contain at least 100 entries, most of whose values will be zero.

There are alternatives that avoid all these shortcomings but are significantly less elegant and therefore more error prone. For example, the `at` member function looks a lot like `operator[]` and has none of the above shortcomings, but missing keys are handled by throwing exceptions, making this option impractical for situations other than when the key is almost certain to exist. Moreover, a `try-catch` block within a loop is rarely a clean way to structure code:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
{
    try {
        largest = std::max(largest, theMap.at(i));
    } catch (const std::out_of_range&) { }
}
```

The above code would work with a `const` map, would ignore missing elements (rather than treating them as zeros), and would not populate the map with useless entries, but many programmers would argue that the loop is inelegant at best. In most C++ implementations, this code would be extremely inefficient unless key misses are rare.

The other obvious alternative uses the `find` member function:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
{
    auto iter = theMap.find(i);
    if (iter != theMap.end())
        largest = std::max(largest, iter->second);
}
```

This version of the loop is only slightly more verbose than our starting point and avoids all the issues, but using iterators, needing to call *two* member functions (`find` and `end`), and having to extract the second member of the element pair for what should be a *simple* operation increases the cognitive load on both the programmer and the reader. Moreover, a generic use of `find` can yield a subtle bug. Consider the following function template, `f`:

```
template <class Key, class Value>
void f(const Key& k, const std::map<Key, Value>& aMap)
{
    Value obj = some-default-obj-value-expression;
    auto iter = aMap.find(k);
    if (iter != aMap.end())
        obj = iter->second;
    // code that uses `obj` ...
}
```

An instantiation of `f` will not compile unless `Value` is copy assignable. Worse, unless tested with a nonassignable type, the bug could go undetected for a long time. One fix would be initialize `obj` in a single expression:

```
Value obj = aMap.count(k) ? aMap.at(k) : some-default-obj-value-expression;
```

While correct, this solution involves two lookup operations: one for the `count` and one for the `at`. A better fix requires a bit more code:

```
auto iter = aMap.find(k);
Value obj = iter != aMap.end() ? iter->second : some-default-obj-value-expression;
```

Note that the last solution again involves iterator, pair, and a conditional expression, which is a far cry from the simplicity of `operator[]`.

Let's contrast these less-than-ideal map lookups to dictionary lookups in another language and consider one way to write the largest-value computation in Python:

```
inf = float("inf")
largest = -inf
for i in range(1, 101):
    largest = max(largest, theMap.get(i, -inf))
```

The `get` member of Python's dictionary type looks up the supplied key (first argument). If the key exists in the dictionary, `get` returns the corresponding value; otherwise, `get` returns the specified alternative value (second argument).

In this paper, I propose `get` member functions for `std::map`, `std::unordered_map`, and `std::flat_map` similar to the `get` member in Python dictionaries. Because C++ does not have a

None value like Python's, `get` returns an `optional`, instead, and delegates the construction of an alternative return value to the `optional` observer members.

§ 4 Proposed Feature

§ 4.1 Overview

What's desired is a simple expression that, given a key, returns the mapped value if the key exists in a specific map and a user-supplied *alternative value* if the key does not exist. The proposed feature is a `get` member function for map-like containers that returns `optional<T&>` (or, for a `const` map, `optional<const T&>`):

```
template<class Key, class T, class Compare = less<Key>,  
         class Allocator = allocator<pair<const Key, T>>>  
class map {  
    // ...  
    constexpr optional< mapped_type&> get(const key_type& k);  
    constexpr optional<const mapped_type&> get(const key_type& k) const;  
    //...  
};
```

These functions depend on having an `optional` template that can be instantiated with reference types, as proposed in [P2988R12], which is expected to be added to the working draft in the June 2025 meeting in Sofia.

Using this feature, the earlier example could be written almost as simply as the Python version:

```
constexpr double inf = std::numeric_limits<double>::infinity();  
double largest = -inf;  
for (int i = 1; i <= 100; ++i)  
    largest = std::max(largest, theMap.get(i).value_or(-inf));
```

§ 4.2 Not Proposed: Enhancements to `optional::value_or`

To maximize the usefulness and convenience of the proposed `get` function, earlier revisions of this paper also proposed enhancements to `optional<T>::value_or` and `optional<T&>::value_or`, allowing (but not requiring) the caller to specify a return type other than `T`, and giving it a variadic argument list comprising constructor arguments for the return value. Although such an extension would be useful, it can also be provided through a non-member function that is expected to be in the next revision of [P1255] by Steve Downey. Since that change does not directly relate to the subject of this proposal, there is no reason to combine it with this paper. You can see the proposed extensions in a previous revision of this paper, [P3091R2].

§ 5 Before and After Comparisons

The following table shows how operations are simplified using the proposed new member functions. In these examples, `K`, `T`, and `U` are object types; `m` is an object of (possibly `const`) `std::map<K, T>`, `unordered_map<K, T>`, or `flat_map<K, T>`; `k` is a value compatible with `K`; `v` is an lvalue of type (possibly `const`) `T`; and `a1..aN` are arguments that can be used to construct an object of type `T`.

Before	After
<pre> auto iter = m.find(k); T x = iter == m.end() ? T{} : iter->second; </pre>	<pre> T x = m.get(k).value_or(T{}); </pre>
<pre> auto iter = m.find(k); T x = iter == m.end() ? T(a1...aN) : iter->second; </pre>	<pre> T x = m.get(k).value_or(T(a1...aN)); </pre>
<pre> auto iter = m.find(k); T& x = iter == m.end() ? v : iter->second; </pre>	<pre> T& x = std::reference_or(m.get(k), v); // assuming P1255 </pre>
<pre> map<K, vector<U>> m{ ... }; auto iter = m.find(k); span<U> x = iter == m.end() ? span<U>{} : iter->second; </pre>	<pre> map<K, vector<U>> m{ ... }; span<U> x = value_or<span<U>>(m.get(k)); // assuming P1255 </pre>
<pre> map<K, vector<U>> m{ ... }; const array<U, N> preset{ ... }; auto iter = m.find(k); span<const U> x = iter == m.end() ? span<const U>(preset) : iter->second; </pre>	<pre> map<K, vector<U>> m{ ... }; const array<U, N> preset{ ... }; span<const U> x = value_or<span<const U>>(m.get(k), preset); // assuming P1255 </pre>
<pre> unordered_map<K, U*> m{ ... }; if (auto iter = m.find(k); iter != m.end()) { U* p = iter->second; // use p ... } </pre>	<pre> unordered_map<K, U*> m{ ... }; if (U* p = m.get(k).value_or((U*) nullptr); p) { // use p ... } // OR unordered_map<K, U*> m{ ... }; m.get(k).and_then([=](U* p) { // use p ... }); // OR unordered_map<K, U*> m{ ... }; for (U* p : m.get(k)) { // use p ... } </pre>
<pre> if (auto iter = m.find(k); iter != m.end()) { T& r = iter->second; } </pre>	<pre> if (auto q = m.get(k); q) { T& r = *q; // use r ... } </pre>

Before	After
<pre>// use r ... }</pre>	<pre>// OR for (auto& r : m.get(k)) { // use r ... }</pre>

§ 6 Alternatives Considered

§ 6.1 Other Names for get

The name `map::get` is borrowed from the Python dictionary member of the same name. Other names considered were `get_optional`, `lookup`, and `lookup_optional`. The `get` name was selected for brevity and familiarity, but `lookup` is also concise and is a reasonable alternative.

In Sofia on 2025-06-19, there was weak consensus in LEWG to keep the name `get`:

POLL: Rename `.get` to `.lookup`

SF	F	N	A	SA
1	3	6	2	6

§ 6.2 Build Get-Value-or-Return-Alternative Functionality Directly into map

Version R0 of this paper proposed `get`, and `get_ref` member functions that would look up a key and return the corresponding value (or a reference to the corresponding value) or else construct an alternative from the nonkey arguments without, involving `optional<T&>`:

```
// return a value  
template <class U = remove_cvref_t<T>, class... Args>  
    U get(const key_type& key, Args&&... args);  
template <class U = remove_cvref_t<T>, class... Args>  
    U get(const key_type& key, Args&&... args) const;  
  
// return a reference  
template <class Arg>  
    auto get_ref(const key_type& key, Arg&& ref)          -> common_reference_t<  
        mapped_type&, Arg>;  
template <class Arg>  
    auto get_ref(const key_type& key, Arg&& ref) const -> common_reference_t<const  
        mapped_type&, Arg>;
```

The following table shows the usage of the R0 design compared to the currently proposed design.

R0 Design (get Returns T)	Current Design (get Returns optional<T&>)
<pre>auto tval = theMap.get(k);</pre>	<pre>auto tval = theMap.get(k).value_or(T{});</pre>

R0 Design (get Returns T)	Current Design (get Returns optional<T&>)
<pre>auto tval = theMap.get(k, cT1, cT2);</pre>	<pre>auto tval = theMap.get(k).value_or(T(cT1, cT2));</pre>
<pre>auto& tref = theMap.get_ref(k, lvalue);</pre>	<pre>// per P1255 auto& tref = reference_or(theMap.get(k), lvalue);</pre>
<pre>auto uval = theMap.get<U>(k, cU1, cU2);</pre>	<pre>// per P1255 auto uval = value_or<U>(theMap.get(k), cU1, cU2);</pre>
<pre>if (auto opt = theMap.get<std::optional<T&>>(k); opt) { auto& ref = *opt; // ... }</pre>	<pre>if (auto opt = theMap.get(k); opt) { auto& ref = *opt; // ... }</pre>

Advantages of the R0 Design Over the Current Design

- R0 provides more concise and readable calls to get (and get_ref) in many cases.
- The desired return-value-if-present-else-return-alternative functionality is all specified in one place in the R0 design, exactly where a beginner would look for it.
- R0 map and unordered_map specification changes would be independent of optional<T&>, thus decoupling this paper from [P2988R12].

Advantages of the Current Design Over the R0 Design

- The current get provides a direct indication, via a disengaged optional return value, that the key was not found.
- By returning optional, the current design can leverage all of the functionality of optional, including value_or and monadic operations. Any future improvements to optional could be accessed by users of map::get without modifying map. In particular, all of the features in [P1255] could be applied to the return value of map::get, including a std::reference_or function.
- The current version has a single observer (get) compared to two observers for the R0 design (get and get_ref). Since each observer requires four overloads (const and non-const, each having a key_type or templated K parameter), the interface simplification is notable.
- The current specification of get is simple to specify and implement, making get easy to add to nonstandard map-like containers and new standard containers such as flat_map.
- The current interface can be applied not only to associative map-like containers, but to any indexable container, including vector and deque, should the committee choose to do that.

During the [2024-02-27 LEWGI \(SG18\) telecon](#), unanimous consent was achieved for get returning optional<T> (known as the Alternative Design in [\[P3091R0\]](#)):

POLL: The alternative design with a smaller container API with extensions to std::optional should be pursued.

SF	WF	N	WA	SA
6	4	0	0	0

§ 6.3 Free Functions Instead of Members

Providing the functionality described in this paper is possible using namespace-scope functions, without modifying std::map and std::unordered_map:

```
template <class Map, class K, class... Args>
    typename optional<typename Map::mapped_type&> get(Map& m, const K& k);
template <class Map, class K, class... Args>
    typename optional<const typename Map::mapped_type&> get(const Map& m, const K&
        k);

auto x = get(m, k).value_or(v);
```

One benefit to this approach is that the namespace-scoped get template can handle any *map-like* dictionary type (i.e., a type having a mapped_type and a find method that returns an iterator pointing to a key-value pair). Such an approach, however, has disadvantages.

- A global function is less intuitive because it puts get outside of the map interface.
- The short name can cause ADL issues.
- The *map-like* concept is novel and not used elsewhere in the Standard. Whether many classes model this concept and would therefore benefit from the generality is unclear.

§ 7 Implementation Experience

An implementation, with tests and usage examples, can be found at https://github.com/phalpern/WG21-halpern/tree/P3091/P3091-map_lookup.

Some of the functionality can be found in Meta's [\[Folly\]](#) library.

§ 8 Wording

This wording is relative to the July 2024 working draft, [\[N5008\]](#).

§ 8.1 Feature-Test Macros

To the list in 17.3.2 [\[version.syn\]](#)¹, add:

```
#define __cpp_lib_map_get yyyymmL // also in <map>, <unordered_map>,
<flat_map>
```

§ 8.2 std::map Changes

In 23.4.3.1 [\[map.overview\]](#)/2, insert the get element-access members:

```
// 23.4.3.3 \[map.access\], element access
constexpr mapped_type& operator[](const key_type& x);
constexpr mapped_type& operator[](key_type&& x);
template<class K> constexpr mapped_type& operator[](K&& x);
constexpr mapped_type& at(const key_type& x);
const constexpr mapped_type& at(const key_type& x) const;
template<class K> constexpr mapped_type& at(const K& x);
template<class K> const constexpr mapped_type& at(const K& x) const;
constexpr optional<mapped_type&> get(const key_type& x) noexcept;
constexpr optional<const mapped_type&> get(const key_type& x) const
noexcept;
template<class K> constexpr optional<mapped_type&> get(const K& x)
noexcept;
template<class K> constexpr optional<const mapped_type&> get(const K& x)
const noexcept;
```

At the end of 23.4.3.3 [\[map.access\]](#), add these descriptions:

```
constexpr optional<mapped_type&> get(const key_type& x) noexcept;
constexpr optional<const mapped_type&> get(const key_type& x) const
noexcept;
template<class K> constexpr optional<mapped_type&> get(const K& x)
noexcept;
template<class K> constexpr optional<const mapped_type&> get(const K& x)
const noexcept;
```

Constraints: For the third and fourth overloads, the *qualified-id* `Compare::is_transparent` is valid and denotes a type.

Preconditions: The expression `find(x)` is well formed and has well-defined behavior.

Returns: `find(x)→second` if `find(x) == end()` is false, otherwise `nullopt`.

Complexity: Logarithmic

§ 8.3 `std::unordered_map` Changes

In 23.5.3.1 [\[unord.map.overview\]](#)/3, insert the get element-access members:

```
// 23.5.3.3 \[unord.map.elem\], element access
constexpr mapped_type& operator[](const key_type& x);
constexpr mapped_type& operator[](key_type&& x);
template<class K> constexpr mapped_type& operator[](K&& x);
constexpr mapped_type& at(const key_type& x);
const constexpr mapped_type& at(const key_type& x) const;
template<class K> constexpr mapped_type& at(const K& x);
template<class K> const constexpr mapped_type& at(const K& x) const;
```

```
constexpr optional<mapped_type&>      get(const key_type& x) noexcept;
constexpr optional<const mapped_type&> get(const key_type& x) const
noexcept;
template<class K> constexpr optional<mapped_type&>      get(const K& x)
noexcept;
template<class K> constexpr optional<const mapped_type&> get(const K& x)
const noexcept;
```

At the end of 23.5.3.3 [unord.map.elem], add these descriptions:

```
constexpr optional<mapped_type&>      get(const key_type& x) noexcept;
constexpr optional<const mapped_type&> get(const key_type& x) const
noexcept;
template<class K> constexpr optional<mapped_type&>      get(const K& x)
noexcept;
template<class K> constexpr optional<const mapped_type&> get(const K& x)
const noexcept;
```

Constraints: For the third and fourth overloads, the *qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

Preconditions: The expression `find(x)` is well formed and has well-defined behavior.

Returns: `find(x)→second` if `find(x) == end()` is false, otherwise `nullopt`.

§ 8.4 `std::flat_map` Changes

In 23.6.8.2 [flat.map.defn], , insert the get element-access members:

```
// 23.6.8.6 [flat.map.access], element access
constexpr mapped_type& operator[](const key_type& x);
constexpr mapped_type& operator[](key_type&& x);
template<class K> constexpr mapped_type& operator[](K&& x);
constexpr mapped_type& at(const key_type& x);
constexpr const mapped_type& at(const key_type& x) const;
template<class K> constexpr mapped_type& at(const K& x);
template<class K> constexpr const mapped_type& at(const K& x) const;
constexpr optional<mapped_type&>      get(const key_type& x) noexcept;
constexpr optional<const mapped_type&> get(const key_type& x) const
noexcept;
template<class K> constexpr optional<mapped_type&>      get(const K& x)
noexcept;
template<class K> constexpr optional<const mapped_type&> get(const K& x)
const noexcept;
```

At the end of 23.6.8.6 [flat.map.access], add these descriptions:

```
constexpr optional<mapped_type&>      get(const key_type& x) noexcept;
constexpr optional<const mapped_type&> get(const key_type& x) const
noexcept;
template<class K> constexpr optional<mapped_type&>      get(const K& x)
noexcept;
template<class K> constexpr optional<const mapped_type&> get(const K& x)
const noexcept;
```

Constraints: For the third and fourth overloads, the *qualified-id* `Compare::is_transparent` is valid and denotes a type.

Preconditions: The expression `find(x)` is well formed and has well-defined behavior.

Returns: `find(x)→second` if `find(x) == end()` is false, otherwise `nullopt`.

Complexity: Logarithmic

§ 9 Acknowledgments

Thanks to Tomasz Kamiński for pushing me on the `optional<T&>` approach.

Thanks to Steve Downey for working with me to harmonize P2988 and P1255 with this paper.

Thanks to Lori Hughes for editing support.

§ 10 References

[Folly] Meta. `folly/ folly/MapUtil.h`.

<https://github.com/facebook/folly/blob/323e467e2375e535e10bda62faf2569e8f5c9b19/folly/MapUtil.h#L35-L71>

[N5008] Thomas Köppe. 2025-03-15. Working Draft, Programming Languages — C++.

<https://wg21.link/n5008>

[P1255] Steve Downey. A view of 0 or 1 elements: `views::nullable` And a concept to constrain maybes.

<http://wg21.link/P1255>

[P2988R12] Steve Downey, Peter Sommerlad. 2025-04-04. `std::optional<T&>`.

<https://wg21.link/p2988r12>

[P2988R4] Steve Downey, Peter Sommerlad. 2024-04-16. `std::optional<T&>`.

<https://wg21.link/p2988r4>

[P2988R7] Steve Downey, Peter Sommerlad. 2024-09-10. `std::optional<T&>`.

<https://wg21.link/p2988r7>

[P3091R0] Pablo Halpern. 2024-02-06. Better lookups for ``map`` and ``unordered_map``.

<https://wg21.link/p3091r0>

[P3091R2] Pablo Halpern. 2024-05-22. Better lookups for ``map`` and ``unordered_map``.

<https://wg21.link/p3091r2>

1. All citations to the Standard are to working draft N5008 unless otherwise specified. ↩